UNIX PERMISSIONS: A SIMPLIFIED ACL

Before modern access control systems evolved, Unix provided one of the earliest and most widely adopted forms of resource-based access control. Although it doesn't use the term *Access Control List*, the Unix file permission model is conceptually a very simple form of an ACL.

Each file or directory has three categories of principals:

- Owner: generally the user who created the file.
- Group: a specific group of users linked to the file.
- Others: everyone else on the system.

Permissions are assigned separately to each of these categories using a fixed set of actions: read (r), write (w), and execute (x). Here's an example file listing for a shell script named `deploy.sh`:

```
-rwxr-xr--  1 alice  devs  deploy.sh
```

This indicates:

- `alice` (the owner) can read, write, and execute the script (denoted by the `rwx`).
- Members of the `devs` group can read and execute it, but they cannot modify it (denoted by the `r-x`).
- Everyone else can only read the file (denoted by the `r--`).

Unlike modern ACLs, which can define many named principals with varying permissions, Unix permissions are limited to three fixed roles and a predetermined set of actions. There's no way to assign access to arbitrary users outside of the owner/group/others structure.

Still, Unix permissions reflect the core ACL concept: a list of principals and their permitted actions is directly attached to the resource and evaluated at the time of access.